

FLASHMASK: FlashAttention 的高效且丰富的掩码扩展

Guoxia Wang*, JinleZeng*, Xiyuan Xiao, Siming
Wu, Jiabin Yang, Lujing Zheng, Zeyu Chen,
Jiang Bian, Dianhai Yu†, Haifeng Wang

Baidu
Inc. 2024年10
月2日

{wangguoxia, zengjinle, yudianhai}@baidu.com

摘要

朴素注意的计算与内存需求随序列长度 N 二次增长, 这给在 Transformer 模型中处理长序列带来了巨大挑战。FlashAttention 通过消除 $O(N^2)$ 的内存依赖并借助面向 IO 的内存优化降低注意延迟, 从而缓解了这些挑战。然而, 其对某些注意掩码类型的原生支持有限, 且并不能从根本上满足更复杂的掩码需求。以往方法转而使用内存复杂度为 $O(N^2)$ 的稠密掩码, 导致低效。本文提出 FLASHMASK, 这是对 FlashAttention 的扩展, 引入了按列的注意掩码稀疏表示。该方法可高效表示多种掩码类型, 并促进优化内核实现的开发。通过采用这种新表示, FLASHMASK 实现了线性内存复杂度 $O(N)$, 适用于建模长上下文序列。此外, 这种表示使内核优化能够利用注意掩码中的稀疏性消除不必要的计算, 在不牺牲计算准确率的前提下, 提高计算效率。我们在 LLMs 的微调与对齐训练 (如 SFT、LoRA、DPO 和 RM) 中评估了 FLASHMASK 的性能。与现有的 FlashAttention 稠密方法相比, FLASHMASK 带来显著的吞吐量提升, 端到端加速比为 1.65 倍至 3.22 倍。此外, 我们的内核级对比表明, 在内核 TFLOPs/s 指标上, FLASHMASK 相比最新的同类方法 FlexAttention 提升了 12.1% 至 60.7%, 并在 A100 GPU 上达到了理论最大 FLOPs/s 的 37.8% 至 62.3%。代码已在 PaddlePaddle¹ 开源, 并集成进 PaddleNLP², 支持参数规模超过 1000 亿的模型, 能够处理最长 128K Token 的上下文。

1 引言

Transformer 架构 Vaswani 等人 (2017年) 已成为跨自然语言处理 (NLP)、计算机视觉 (CV) 和多模态应用中广泛任务的基础模型。其有效性的核心在于注意机制, 使模型能够关注输入数据的相关部分。在标准注意机制中, 注意权重被计算为查询与键向量之间的缩放点积, 如公式 (1) 所示。为实现复杂逻辑, 掩码 M 可以在应用 Softmax 函数之前添加到 QK^T 项, 通过将某些元素设置为 $-\infty$ 来控制标记可见性。

*同等贡献。

†通讯作者。¹<https://github.com/PaddlePaddle/Paddle>²
<https://github.com/PaddlePaddle/PaddleNLP>

$$\text{Attention}(Q, K, V, M) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \quad (1)$$

在 NLP 任务中，大型语言模型（LLM）的训练通常可分为两个主要阶段：预训练（PreTraining）和后训练（PostTraining）。在预训练期间，会采用不同的掩码策略来引导模型的学习过程。例如，GPT 风格 Radford（2018年）模型使用单向因果掩码，如图1（a）（1）所示；而 T5 风格 Raffel 等人（2020年）模型采用单向与双向掩码的组合，如图1（a）（9）所示。后训练通常包括有监督微调（SFT）Iyer 等人（2022年）；Chung 等人（2024年）；Hu 等人（2021年）；Liu 等人（2024a）；Chen 等人（2023年），直接偏好优化（DPO）Rafailov 等人（2024年）；Dong 等人（2023年）；Liu 等人（2023年）；Ethayarajh 等人（2024年）；Liu 等人（2024b），以及基于人类反馈的强化学习（RLHF）中的奖励模型（RM）训练 Schulman 等人（2017年）；Shao 等人（2024年），也会根据特定任务采用多种掩码技术，如图1（a）（3）和（5）所示。

掩码策略日益复杂多样，对注意机制的效率提出了重大挑战。具体而言，朴素注意机制在计算与内存需求上呈二次增长，记为 $O(N^2)$ ，其中 N 表示序列长度。随着模型扩展到更长的序列，在如 GPT-4 Achiam et al.（2023年）、Claude Anthropic（2024年）和 Gemini Reid et al.（2024年）等先进系统中达 128K 到 1M Token，这种二次依赖变得难以承受，需要更高效的计算方法。带掩码的注意计算的内存负载也呈二次增长，进一步加剧了在不同任务中管理各种掩码配置的挑战。

近期的工作，如 Memory Efficient Attention（MEA）Rabe & Staats（2021年）和 FlashAttention Dao et al.（2022年）；Dao（2023年），通过降低内存开销和注意延迟，在解决这些问题方面取得了进展。尤其是 FlashAttention，消除了 $O(N^2)$ 的内存依赖，并通过面向 IO 的内存读/写优化降低了注意延迟。然而，尽管 FlashAttention 原生支持某些掩码类型且不增加内存开销，但其对更复杂掩码需求的支持仍然有限。

在本文中，我们提出了 **FLASHMASK**，这是 FlashAttention 的一种扩展，利用了一种新颖的列式注意掩码表示。该方法能够高效处理更广泛的掩码类型，同时不牺牲计算准确率。FLASHMASK 实现了线性内存复杂度，同时启用可减少不必要计算的内核优化，从而带来显著的计算加速并提升训练效率。

我们的贡献体现在三个方面：

1. 我们提出了一种新颖的列式稀疏掩码表示，可高效适配更广泛的掩码类型，从而实现更灵活的注意机制。
2. 我们通过集成优化的内核实现扩展了 FlashAttention 的掩码能力，在不牺牲计算准确率的前提下确保高计算效率。
3. 我们在多种注意掩码类型和模型上验证了 FLASHMASK 的有效性，凸显其在大规模大型语言模型训练中的通用性与稳健性。FLASHMASK 显著降低了计算和内存开销，大幅提升其在长上下文建模中的适用性。

2 背景

2.1 注意掩码类型

基于Transformer的模型在各种任务上展现出非凡的通用性，每种任务都能从不同的注意掩码类型中受益，如图1所示。因果掩码 主要用于自回归模型以预测序列中的下一个标记，确保每个标记只关注先前的标记，避免来自未来标记的信息泄漏 Vaswani 等人（2017年）。滑动窗口掩码 通过允许标记关注邻近标记的固定大小窗口来捕获局部上下文，在捕获局部依赖能力与计算效率之间取得平衡 Beltagy 等人（2020年）。因果文档掩码，用于诸如高效序列打包与 in-batch/in-tokens 技术的方法中，可在不

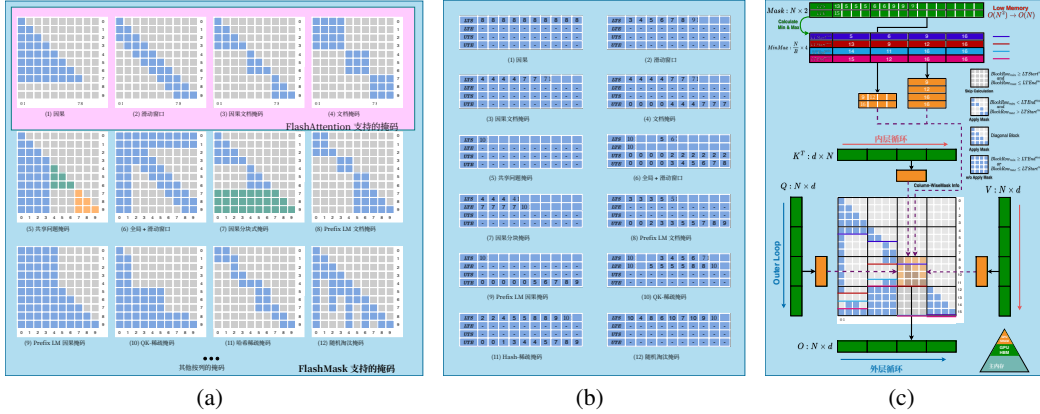


图1: FLASHMASK 概览。(a) FLASHMASK 支持的掩码类型, (b) FLASHMASK 中按列的稀疏表示, (c) FLASHMASK 的高效实现。

牺牲性能的情况下加速大型语言模型, 通过确保标记只关注同一文档内先前的标记来实现 Krell 等人 (2021年); Iyer 等人 (2022年); Dubey 等人 (2024年)。文档掩码, 或称双向注意, 允许标记关注同一文档内的所有其他标记, 便于从两个方向学习上下文, 并被广泛应用于诸如 BERT 以及诸如 NaViT 的视觉 Transformer 模型中 Devlin 等人 (2018 年); Dehghani 等人 (2024 年)。

共享问题掩码 用于奖励模型 (RM) 和 Direct Preference Optimization (DPO) 模型, 允许多个答案共享一个问题, 从而消除冗余计算并加速训练 Ouyang 等 (2022年)。全局 + 滑动窗口掩码将全局注意与滑动窗口注意相结合, 其中全局 Token 关注所有 Token, 而其他 Token 使用滑动窗口掩码, 能够有效处理既需要全局上下文又需要局部细节的任务 Zaheer 等 (2020年)。

因果分块掩码, 主要用于 in-context learning, 将序列划分为若干块, 其中示例仅在小块内关注邻近示例, 而测试示例可以关注所有示例, 从而能够研究在长上下文任务中的模型性能改进 Bertsch 等 (2024年)。前缀 LM 因果掩码 针对语言建模任务进行定制, 允许一个前缀关注所有 Token, 以基于该前缀生成连贯文本 Raffel 等 (2020年)。前缀文档掩码 将这一概念扩展到多个文档, 其中每个文档中的前缀关注该文档内的所有 Token, 但不跨文档关注。

QK-Sparse Mask 通过将查询-键对稀疏化来优化自注意力, 在保持性能的同时降低计算负载, 这对大规模模型尤其有益 (Kitaev 等, 2020年)。 *Hash-Sparse Mask* 采用局部敏感哈希将序列划分为更小的片段, 从而为长序列实现高效的稀疏注意 (Kitaev 等, 2020年)。最后, *RandomEviction Mask* 通过在训练期间随机屏蔽标记来引入随机性, 有助于泛化, 并模拟 Key-Value (KV) 缓存驱逐流程, 从而在不发生内存溢出的情况下处理长序列 (Chen 等, 2024年)。

2.2 注意掩码支持

注意机制是基于Transformer的模型的基础, 各种掩码类型能够实现不同的注意模式。如公式 2 所示的标准注意机制, 通过稠密掩码矩阵支持任意掩码类型:

$$S = \frac{QK^T}{\sqrt{d_k}} \in \mathbb{R}^{N \times N}, \quad P = \text{Softmax}(S + M) \in \mathbb{R}^{N \times N}, \quad O = PV \in \mathbb{R}^{N \times d}, \quad (2)$$

其中 $Q, K, V \in \mathbb{R}^{N \times d}$ 为输入序列, $M \in \mathbb{R}^{N \times N}$ 为注意掩码, N 为序列长度, d 为头维度。掩码 M 通过与 S 逐元素相加来调制标记可见性。尽管这种方法支持任意掩码类型, 但其内存复杂度为 $O(N^2)$, 从而限制了其在长序列上的可扩展性。

FlashAttention Dao 等人 (2022年) ; Dao (2023年) 通过 IO 感知的读/写操作和分块技术解决了这一限制, 取消了对中间表示 $S \in \mathbb{R}^{N \times N}$ 和显式掩码 M 的需求。然而, FlashAttention 在其内核中仅支持预定的掩码模式, 例如因果、滑动窗口、因果文档以及文档掩码, 如图1所示。

xFormers Lefaudeux 等人 (2022年) 扩展了 FlashAttention 的能力, 提供对带对角偏移的掩码的支持。它使用累积的序列长度表示文档掩码, 实现了 $O(N)$ 的内存复杂度。

FlexAttention He 等人 (2024年) 基于深度学习编译器技术引入了一种更灵活的掩码描述方法。通过将块掩码与基于表达式的描述相结合, 它可以支持任意掩码类型。尽管该方法通过基于块的处理显著降低了内存开销, 但其内存复杂度仍为 $O(\frac{N^2}{B_r B_c})$ 。

我们提出的方法 FLASHMASK 扩展了 FlashAttention 的掩码支持能力。它引入了一种灵活的、按列稀疏的掩码表示, 覆盖了大多数主流的 Transformer 建模需求。如图1(b)所示, FLASHMASK 按列表达需要被屏蔽的区间, 实现了 $O(N)$ 的内存复杂度。该方法在掩码灵活性与计算效率之间架起桥梁, 为大规模 Transformer 模型中的注意机制提供了更通用的解决方案。

2.3 注意优化技术

如公式 2 所述, 注意机制在计算和内存方面带来了显著挑战, 尤其是在对 QK^T 的计算中。随着序列长度 N 增加, 得到的注意分数矩阵按平方增长, 导致复杂度为 $O(N^2)$ 。为了解决这一可扩展性问题, 研究者提出了多种优化技术, 重点关注内存效率和计算速度。

内存高效注意 (MEA) Rabe & Staats (2021年) 标志着模型训练优化方面的一个重要进展。通过利用 Online Softmax Milakov & Gimelshein (2018年) 和分块技术, MEA 将内存需求从 $O(N^2)$ 降至 $O(\sqrt{N})$, 使在现有硬件限制内能够使用更大的模型或更长的序列长度。在此基础上, FlashAttention Dao 等 (2022年) ; Dao (2023年) 通过面向 IO 的内存读/写优化来降低注意延迟。在计算过程中利用 tiling 分块技术, FlashAttention 实现了 $O(N)$ 的内存开销, 在不需要自定义掩码的任务中尤为有效。此外, FlashAttention 扩展为 Block-Sparse FlashAttention, 引入二维块掩码矩阵表示, 用于指示被掩蔽的 tiling 块。这一创新允许跳过被掩蔽块的计算, 从而加速过程。

对于需要特定注意掩码的场景, 已经出现了若干定制化方案。Pagliardini 等人 (2023年) 提出的稀疏因果 FlashAttention (SCFA) 将 FlashAttention 扩展到因果注意结构中的 QK-Sparse 和 Hash-Sparse 场景。SCFA 使用原始未压缩张量中查询和键的索引来描述掩码, 从而能够省略对被掩码块的计算, 并提升计算效率。He 等人 (2024年) 的 FlexAttention 利用编译器技术简化掩码注意的实现, 利用注意掩码中的稀疏性跳过某些被掩码的块, 从而获得更高的速度。然而, 仍有优化空间, 尤其是对于复杂的掩码模式。

我们提出的方法 FLASHMASK 基于这些进展, 支持定制的复杂注意掩码。FLASHMASK 将内存复杂度从 $O(N^2)$ 降至 $O(N)$, 同时利用注意掩码中的稀疏性以跳过被掩蔽的块。通过严谨的工程优化, FLASHMASK 相较于 FlexAttention 实现了更优的计算速度, 尤其在具有复杂掩码需求的任务中。通过将现有方法的优势与新的优化技术相结合, FLASHMASK 在注意机制效率方面取得了重大进展, 在大规模 Transformer 模型中同时解决内存约束和计算速度问题。

3 观察

在当前训练基于Transformer的模型的范式中，如第2.1节所述并在图1(a)中所示，注意机制可以根据其因果性被大致分类。这些表示涵盖了训练场景中遇到的大多数掩码类型。考虑注意得分矩阵 S ，其中每个元素 S_{ij} 表示第 i 个查询标记对第 j 个键标记的注意。从键标记的角度来看，我们观察到查询标记如何对每个键标记进行注意的一个关键模式。

我们的关键观察是：查询 Token 无法关注某些键 Token 的现象具有连续性。这种连续性使我们能够将二维稠密掩码 M 转变为一种更为紧凑的一维表示，即使用行索引区间，如图1(b)所示。形式上，我们可以将该变换表示为：

$$M_j = [start_j, end_j), \quad \forall j \in \{1, \dots, N\} \quad (3)$$

其中 M_j 表示对第 j 个键 Token 被屏蔽的行索引区间，且 N 是序列长度。

尽管这种按列的一维区间表示可能无法刻画任意的掩码模式，但它能有效覆盖实践中占主导的掩码类型。此外，这种表示具有一个显著优势：它便于在基于分块（tiling）的计算中直接转换为带掩码的块。该转换能够消除不必要的计算，从而提升注意力内核的计算效率。

区间表示的这一概念可以推广到其他形式的连续区间。例如，通过对注意力矩阵转置，我们可以得到使用列索引区间的按行表示。该方法的灵活性使其在保持紧凑表示的同时，能够高效处理各种注意力模式，并有利于优化计算。

4 FLASHMASK：算法与分析

在本节中，我们介绍 FLASHMASK 的新颖按列掩码表示，扩展 FlashAttention 以支持复杂的掩码模式，并对我们的方法进行全面的复杂度分析。

4.1 按列掩码表示

为在因果和双向注意场景中高效处理复杂的掩码模式，我们为 FLASHMASK 提出一种新颖的按列稀疏表示。相对于对角线，将注意得分矩阵划分为左下和右上三角区域。

FLASHMASK 使用四个一维向量来表示掩码：

- **LTS**: LowerTriangular Start - 左下三角中掩码的起始行。
- **LTE**: Lower Triangular End - 左下三角中掩码的结束行。
- **UTS**: Upper TriangularStart - 右上三角形中掩码的起始行。
- **UTE**: Upper Triangular End - 右上三角形中掩码的结束行。

下三角部分中需要被屏蔽的行索引由 $[LTS, LTE)$ 给出，上三角部分由 $[UTS, UTE)$ 给出。

具体而言，每一列由两个掩码区间描述。对于第 j 个标记，位于区间 $[LTS_j, LTE_j) \cup [UTS_j, UTE_j)$ 内的标记不能对其进行注意。例如，如图1(b)(6)所示，对于第五列， $[LTS_5, LTE_5) \cup [UTS_5, UTE_5) = [7, 10) \cup [2, 4)$ 表示第2到4行以及第7到9行被屏蔽。

这种表示具有以下优点：

1. **紧凑性**：它将稠密二维掩码压缩为更高效的一维表示。
2. **灵活性**：它能够覆盖广泛的实际掩码模式，包括因果型、双向以及更复杂的注意机制。
3. **计算效率**：它便于在基于平铺的计算中转换为带掩码的块，从而消除不必要的计算。

4.2 将 FlashAttention 扩展到复杂掩码

我们将 FLASHMASK 的列式掩码表示集成到 FlashAttention-2 算法中，扩展其掩码支持能力。FLASHMASK 的高性能内核实现由两个关键步数组成：

预处理：给定输入的按列稀疏掩码向量，我们首先在高带宽内存（HBM）中沿列方向将其划分为 T_c 个块。对于每个掩码向量，我们在每个块内计算最大值和最小值。这将产生 8 个中间表示向量： $LStart^{max}$, $LStart^{min}$, $LEnd^{max}$, $LEnd^{min}$, $UStart^{max}$, $UStart^{min}$, $UEnd^{max}$, 以及 $UEnd^{min}$ ，每个的大小为 T_c 。

实时块跳过计算：利用这些最小-最大向量，我们可以在内核计算过程中将注意分数矩阵的每个平铺块划分为三类。块掩码类型 T_{block} 的判定如下：

$$T_{block} = \begin{cases} \text{完全掩蔽, if } BlockRow_{min} \geq Start^{max} \text{ and } BlockRow_{max} \leq End^{min} & \text{部分掩蔽, elif} \\ BlockRow_{min} < End^{max} \text{ and } BlockRow_{max} > Start^{min} & \text{未掩蔽, otherwise} \end{cases} \quad (4)$$

这种分类使我们能够跳过完全掩蔽的块，减少对未掩蔽块的计算，并且仅对部分掩蔽的块应用逐元素掩蔽。图1(c)展示了因果场景下的整个内核计算过程，其中 LTS 和 LTE 位于左下三角。算法 1 详细说明了基于 FlashAttention-2 扩展而来的 FLASHMASK 的前向传播过程，其中蓝色阴影部分表示 FLASHMASK 计算。

对于反向传播，FLASHMASK 的列稀疏表示尤其有优势。 dK 和 dV 的计算是列并行的，这使得可以将最大值和最小值高效加载到寄存器中，在块计算过程中进行大量的数据复用，如附录中的算法2所示。

4.3 复杂度分析

我们将注意掩码中的块稀疏性定义为 $\rho = \frac{\alpha}{\lceil \frac{N}{B_r} \rceil \times \lceil \frac{N}{B_c} \rceil}$ ，其中 B_r, B_c 为块大小， α 为完全掩蔽的块数。

空间复杂度：稠密掩码需要 $O(N^2)$ 的空间，而 FLASHMASK 对 $LTS, LTE, UTS, UTE \in \mathbb{R}^N$ 以及 8 个预计算的最小-最大向量 $\in \mathbb{R}^{\lceil \frac{N}{B_c} \rceil}$ 使用 $O(N)$ 的空间。这种显著的内存使用量减少使得可以在更长的序列上进行训练。

内存访问复杂度：稠密掩码在 HBM 上需要 $O(N^2)$ 次内存访问。FLASH- MASK 读取来自 HBM 的 $LTS, LTE, UTS, UTE \in \mathbb{R}^N$ 向量，如算法 1 的第 16 行和第 19 行所示，其中每个 Q_i 都会读取完整的 LTS, LTE, UTS, UTE ，总计 $4 \times T_r \times N$ 次内存访问。这将内存访问量降低到约 $\frac{N^2}{4 \times T_r \times N} = \frac{B_r}{4}$ ，显著提升性能。此外，FLASHMASK 的紧凑表示允许将掩码向量预加载到 SRAM，从而进一步提升内存访问效率。

计算复杂度：虽然标准的注意计算复杂度为 $O(N^2)$ ，但 FLASHMASK 利用注意掩码中的稀疏性，通过跳过完全被掩蔽的块，将其降低到 $O((1 - \rho)T_r T_c)$ 。

这些在空间、内存访问和计算复杂度上的改进，有助于 FLASH- MASK 在处理复杂注意力模式时表现出更优的性能与效率。

4.4 正确性分析

如公式 2 所示，注意矩阵 $P = \text{Softmax}(S + M)$ 的计算涉及将注意得分 S 与掩码 M ，其中被掩蔽的元素被设为 $-\infty$ 。该操作确保这些被掩蔽位置的 Softmax 输出为零，从而在注意中将其有效地忽略。因此，如果一个完整的块被完全掩蔽，则该块得到的输出将全为零。FLASHMASK 通过跳过涉及这些完全被掩蔽块的计算来利用注意掩码中的稀疏性，从而减少计算开销而不改变

算法 1 FlashAttention-2 前向传播, 通过 FLASHMASK 扩展

要求: 矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 位于 HBM 中, 块大小为 B_c, B_r ,

- 1: 将 $\mathbf{Q}$ 划分为 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ 个块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$, 每个大小为 $B_r \times d$, 并将 $\mathbf{K}, \mathbf{V}$ 划分为 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ 个块 $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, 每个大小为 $B_c \times d$ 。
- 2: 将输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 划分为 T_r 个块 $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$, 每个大小为 $B_r \times d$, 并将 $\text{logsumexp } L$ 划分为 T_r 个块 $L_1, \dots, L_{T_r}$, 每个大小为 B_r 。
- 3: 将 $\mathbf{LTS}, \mathbf{LTE}, \mathbf{UTS}, \mathbf{UTE}$ 划分为 T_c 个块 $\mathbf{LTS}_1, \dots, \mathbf{LTS}_{T_c}, \mathbf{LTE}_1, \dots, \mathbf{LTE}_{T_c}, \mathbf{UTS}_1, \dots, \mathbf{UTS}_{T_c}, \mathbf{UTE}_1, \dots, \mathbf{UTE}_{T_c}$, 每个大小为 B_c 。
- 4: 预计算行索引的最小值和最大值 $LTStart_j^{min} = \min(\mathbf{LTS}_j), LTStart_j^{max} = \max(\mathbf{LTS}_j), LTEnd_j^{min} = \min(\mathbf{LTE}_j), LTEnd_j^{max} = \max(\mathbf{LTE}_j), UTStart_j^{min} = \min(\mathbf{UTS}_j), UTStart_j^{max} = \max(\mathbf{UTS}_j), UTEnd_j^{min} = \min(\mathbf{UTE}_j), UTEnd_j^{max} = \max(\mathbf{UTE}_j), \forall j \in \{1, \dots, T_c\}$, 写操作到 HBM。
- 5: **for** $1 \leq i \leq T_r$ **do**
- 6: 加载 $\mathbf{Q}_i$ 从 HBM 到片上 SRAM。
- 7: 片上, 初始化 $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, l(0)_i = (0)_{B_r} \in \mathbb{R}^{B_r}, m(0)_i = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$ 。
- 8: **for** $1 \leq j \leq T_c$ **do**
- 9: **if** $(i-1) \times B_r \geq LTStart_j^{max}$ **and** $i \times B_r \leq LTEnd_j^{min}$ **then**
- 10: Continue // 下三角掩码块跳过计算
- 11: **结束 if**
- 12: **如果** $(i-1) \times B_r \geq UTStart_j^{max}$ **and** $i \times B_r \leq UTEnd_j^{min}$ **then**
- 13: 继续 // 上三角掩码块的跳过计算
- 14: **结束 if**
- 15: 将 $\mathbf{K}_j, \mathbf{V}_j$ 从 HBM 加载到片上 SRAM。
- 16: 在片上, 计算 $\mathbf{S}(j)_i = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ 。
- 17: **if** $i \times B_r > LTStart_j^{min}$ **和** $(i-1) \times B_r < LTEnd_j^{max}$ **then**
- 18: 将 $\mathbf{LTS}_j$ 和 $\mathbf{LTE}_j$ 从 HBM 加载到片上 SRAM。
- 19: 在片上, 应用掩码: $\mathbf{S}(j)_i[x][y] = -\infty, \forall x, y$, 使得 $\mathbf{LTS}_j[y] \leq (i-1) \times B_r + x < \mathbf{LTE}_j[y]$
- 20: **结束 if**
- 21: **if** $i \times B_r > UTStart_j^{min}$ **和** $(i-1) \times B_r < UTEnd_j^{max}$ **then**
- 22: 将 $\mathbf{UTS}_j$ 和 $\mathbf{UTE}_j$ 从 HBM 加载到片上 SRAM。
- 23: 在芯片上, 应用掩码: $\mathbf{S}(j)_i[x][y] = -\infty, \forall x, y$, 使得 $\mathbf{UTS}_j[y] \leq (i-1) \times B_r + x < \mathbf{UTE}_j[y]$
- 24: **结束 if**
- 25: 在芯片上, 计算 $m(j)_i = \max(m(j-1)_i, \text{rowmax}(\mathbf{S}(j)_i)) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}(j)_i = \exp(\mathbf{S}(j)_i - m(j)_i) \in \mathbb{R}^{B_r \times B_c}$ (逐点), $l(j)_i = e^{m(j-1)_i - m(j)_i} l(j-1)_i + \text{rowsum}(\tilde{\mathbf{P}}(j)_i) \in \mathbb{R}^{B_r}$ 。
- 26: 在芯片上, 计算 $\mathbf{O}(j)_i = \text{diag}(e^{m(j-1)_i - m(j)_i}) \mathbf{O}(j-1)_i + \tilde{\mathbf{P}}(j)_i \mathbf{V}_j$ 。
- 27: **结束 for**
- 28: 在芯片上, 计算 $\mathbf{O}_i = \text{diag}(l(T_c)_i)^{-1} \mathbf{O}(T_c)_i$ 。
- 29: 在芯片上, 计算 $L_i = m(T_c)_i + \log(l(T_c)_i)$ 。
- 30: 写操作 $\mathbf{O}_i$ 到 HBM 作为第 i 个块的 $\mathbf{O}$ 。
- 31: 写操作 L_i 到 HBM 作为第 i 个块的 L 。
- 32: **结束 for**
- 33: 返回输出 $\mathbf{O}$ 和 $\text{logsumexp } L$ 。

结果。重要的是, FLASHMASK 与在 FlashAttention 中使用稠密掩码进行的计算在比特级保持数值等价性, 确保没有精度损失。我们的实验评估也证实了这一精确性: 我们验证了端到端训练的损失收敛曲线在比特级上精确对齐 (见第 5.2 节)。

5 实验

本节我们通过一系列实验评估 FLASHMASK 的性能, 旨在展示其不同模型规模和序列长度下的端到端加速与内存效率、在实际场景中的训练收敛性、与注意掩码中的块稀疏性的关系, 以及在各种注意掩码模式下的有效性。所有实验均在配备英伟达 A100-SXM 80G GPU、Intel(R) Xeon(R)

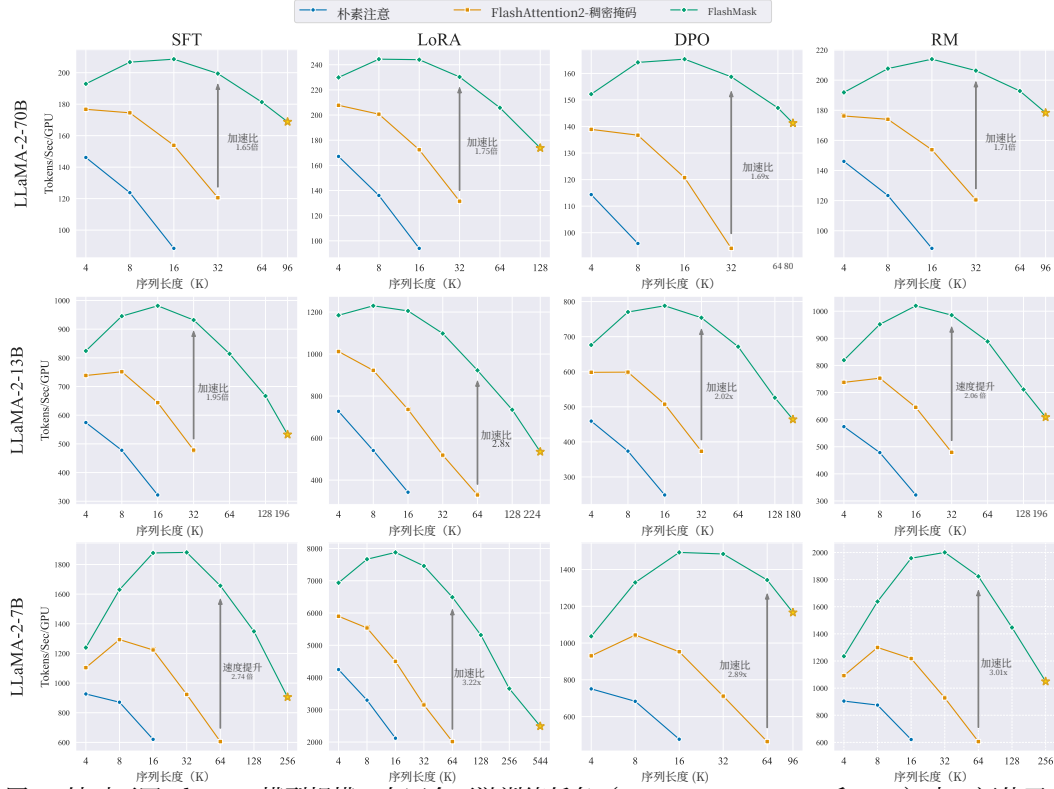


图2: 针对不同 Llama2 模型规模, 在四个下游训练任务 (SFT、LoRA、DPO 和 RM) 中, 评估了在不同序列长度下的端到端训练吞吐量。

Platinum 8350C CPU、CUDA 12.0 以及驱动版本 525.125.06。由于篇幅限制, 有关数据集和超参数设置的详细信息见附录A。

5.1 端到端训练吞吐量

为展示 FLASHMASK 的实际有效性, 我们在三种规模的 Llama-2 模型 (70亿、13B 和 70B) 上, 针对涉及微调与对齐训练的四个下游任务 (SFT、LoRA、DPO 和 RM), 在不同序列长度下评估了端到端训练吞吐量。我们将 FLASHMASK 与两种稠密掩码方法进行了比较。实验结果如图2所示。结果得出两个关键结论。 **更高的吞吐量:** 与具有二次方的内存复杂度的稠密掩码方法相比, FLASHMASK 实现了更高的吞吐量。具体而言, 在 FlashAttention 稠密掩码所支持的最大序列长度下, FLASHMASK 取得了 1.65倍到 3.22倍 的提升。这表明, 在实际应用中, FLASH- MASK 能显著提升大型语言模型的训练吞吐量, 从而降低训练成本。 **线性内存开销:** FLASHMASK 的线性内存开销使其能够支持更长的序列长度。在 Llama-2 70亿 的 LoRA 训练中, FLASHMASK 支持的序列长度可达 544K, 而其他方法受限于 64K。在序列长度为 64K 时, 稠密掩码方法的内存开销达 8GB。图4(b) 展示了内存开销曲线, 凸显了 FLASHMASK 在内存消耗方面的高效性。

5.2 端到端训练收敛性验证

FLASHMASK 的核心创新在于引入按列的稀疏掩码表示, 利用注意掩码中的稀疏性跳过对完全掩蔽块的计算, 从而在不改变算法精度的情况下提升速度。为验证 FLASHMASK 不会损害收敛准确率, 我们在 LLMs 的四个微调与对齐训练任务上, 对 Llama 3.1 (Dubey 等, 2024年) 8B 模型进行了端到端训练实验。

需要注意, 在 CUDA 内核实现中, dQ 的反向计算可能由于累加顺序而引入随机性 (参见算法2第 27行)。因此,

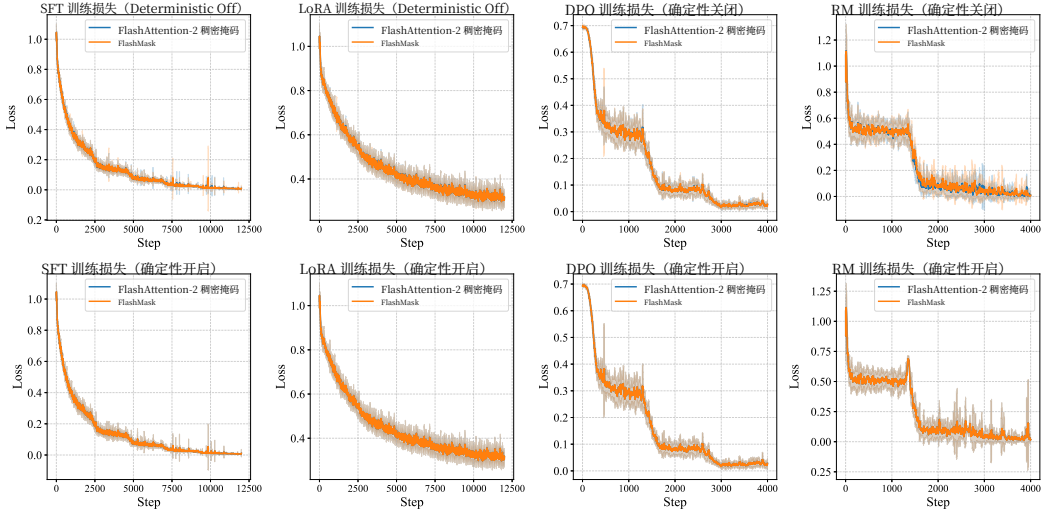


图3：端到端训练损失。

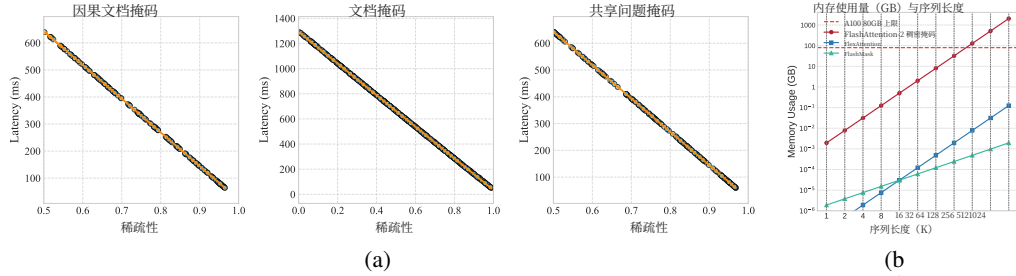


图4：(a) 不同稀疏性水平下的内核执行延迟，(b) 内存使用量，Y 轴使用以 10 为底的对数缩放。

我们在启用和未启用确定性控制的条件下进行了收敛实验。损失曲线如图3所示。在启用确定性控制时，FLASHMASK 与 FlashAttention 稠密掩码的损失曲线严格对齐，表现出完全一致的数值行为。在禁用确定性控制时，两种方法表现出相同的损失收敛趋势。这些结果有力地证明 FLASHMASK 是一种精确的算法，能够保持收敛准确率。

5.3 稀疏性相关实验

FLASHMASK 利用注意掩码中的块稀疏性来跳过对完全掩蔽块的计算，使得计算复杂度与 $O((1 - \rho)T_r T_c)$ 成正比。为验证该关系，我们在三种不同的掩码情形下进行了实验，配置为 bfloat16 数据类型、序列长度 32K、头部维度 128，以及 32 个头。我们采样了具有不同稀疏性水平的数据进行测试。图4 (a) 展示了不同稀疏性水平下的内核执行延迟，表明延迟与稀疏性呈线性关系。

5.4 内核性能比较

为全面评估 FLASHMASK 在常见注意掩码模式下的表达能力和计算效率，我们与 FlexAttention 进行了内核级比较。实验覆盖 12 种不同的掩码案例，序列长度为 8K、32K 和 128K，头维度为 64 和 128，使用 BFloat16 数据类型。令牌总数固定为 128K；随着序列长度变化，相应的批大小也随之变化；同时将隐藏维度固定为 4096，并通过改变头维度来调整注意力头数。FLASHMASK 和 FlexAttention 都利用注意掩码中的稀疏性。我们使用 TFLOPs/s 指标来衡量内核速度。如图5所示，在所有案例中，就前向和反向传播的总 TFLOPs/s 而言，FLASHMASK 均优于 FlexAttention，改进幅度从

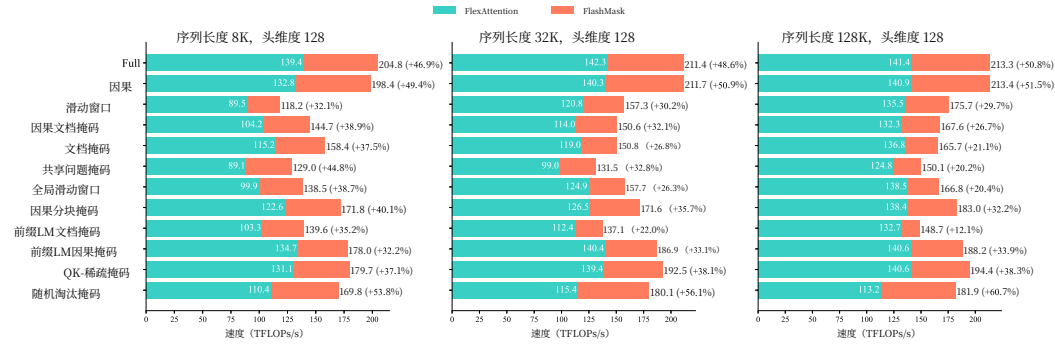


图5: 在 A100-SXM 80G GPU 上的内核前向传播和反向传播速度 (头部维度 128, BF16)。

FlexAttention 使用 PyTorch 2.6.0.dev20240920+cu124。

从 12.1% 到 60.7%。FLASHMASK 在 A100 GPU 上达到理论最大 FLOPs/s 的 37.8% 到 62.3%。

6 局限性与未来方向

尽管 FLASHMASK 显著提升了长序列注意机制的效率, 但仍存在局限性。列式掩码表示将内存复杂度从 $O(N^2)$ 降至 $O(N)$, 为长序列训练带来可观的内存节省, 并能有效捕获最常见的掩码模式。然而, 它无法表示任意掩码, 尤其是单个列内具有不规则被屏蔽区域的掩码。极端情况 (如完全随机的掩码) 在表示与高效计算方面都带来挑战。未来研究应致力于开发在表达能力与计算效率上同时最大化的更复杂稀疏表示, 特别是那些适用于高性能内核分块技术的表示。将 FLASHMASK 扩展以利用更新架构 (例如英伟达的 Hopper) 的特性, 可能进一步提升性能。此外, 尽管我们当前的实现基于 PaddlePaddle Ma 等 (2019年) 框架, 将 FLASHMASK 集成到其他流行的深度学习框架中, 能够扩大其影响力与可用性。这些努力旨在解决当前局限性的同时, 扩大 FLASHMASK 在更广泛任务中的适用性, 推动用于长序列处理的高效 Transformer 模型的持续演进。

7 结论

本文提出了 FLASHMASK, 这是对 FlashAttention 算法的创新扩展, 引入了一种按列的稀疏掩码表示, 以高效处理 Transformer 模型中种类广泛的注意力掩码模式。我们的方法将内存复杂度从 $O(N^2)$ 降至 $O(N)$, 从而能够处理显著更长的序列, 这对现代大语言模型至关重要。通过将该表示融入 FlashAttention 算法并实现优化的内核, FLASHMASK 利用注意力掩码的稀疏性, 在不牺牲计算准确性的情况下跳过对完全掩蔽块的计算。这种策略带来了显著的计算加速: 相较于现有的 FlashAttention 稠密方法, 在大语言模型的微调与对齐训练期间观测到的端到端提升为 1.65 倍至 3.22 倍。此外, FLASHMASK 显著减少了与注意力掩码存储相关的内存开销, 从而进一步支持更长的序列建模。同时, FLASHMASK 在内核 TFLOPs/s 上较最新的对标方法 FlexAttention 提升 12.1% 至 60.7%, 并在 A100 GPU 上达到理论最大 FLOPs/s 的 37.8% 至 62.3%。我们的方法已在大语言模型的下游任务上得到验证, 预计将被业界广泛采用。

致谢

本工作得到北京市科学技术项目 (编号: Z231100010323002) 的资助。

REFERENCES

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Anthropic. Introducing claude. <https://www.anthropic.com/news/introducing-claude>, 2024. Accessed: May 20, 2024.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Amanda Bertsch, Maor Ivgi, Uri Alon, Jonathan Berant, Matthew R Gormley, and Graham Neubig. In-context learning with long-context models: An in-depth exploration. *arXiv preprint arXiv:2405.00200*, 2024.
- Yilong Chen, Guoxia Wang, Junyuan Shang, Shiyao Cui, Zhenyu Zhang, Tingwen Liu, Shuo-huan Wang, Yu Sun, Dianhai Yu, and Hua Wu. NACL: A general and effective KV cache eviction framework for LLM at inference time. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7913–7926, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.acl-long.428>.
- Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. Longlora: Efficient fine-tuning of long-context large language models. *arXiv preprint arXiv:2309.12307*, 2023.
- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.
- Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.
- Mostafa Dehghani, Basil Mustafa, Josip Djolonga, Jonathan Heek, Matthias Minderer, Mathilde Caron, Andreas Steiner, Joan Puigcerver, Robert Geirhos, Ibrahim M Alabdulmohsin, et al. Patch n’pack: Navit, a vision transformer for any aspect ratio and resolution. *Advances in Neural Information Processing Systems*, 36, 2024.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. Raft: Reward ranked finetuning for generative foundation model alignment. *arXiv preprint arXiv:2304.06767*, 2023.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. Kto: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*, 2024.
- Horace He, Driss Guessous, Yanbo Liang, and Joy Dong. FlexAttention: The Flexibility of PyTorch with the Performance of FlashAttention. <https://pytorch.org/blog/flexattention/>, August 2024. Accessed on August 07, 2024.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

- Hamish Ivison, Yizhong Wang, Valentina Pyatkin, Nathan Lambert, Matthew Peters, Pradeep Dasigi, Joel Jang, David Wadden, Noah A. Smith, Iz Beltagy, and Hannaneh Hajishirzi. Camels in a changing climate: Enhancing lm adaptation with tulu 2, 2023.
- Srinivasan Iyer, Xi Victoria Lin, Ramakanth Pasunuru, Todor Mihaylov, Daniel Simig, Ping Yu, Kurt Shuster, Tianlu Wang, Qing Liu, Punit Singh Koura, et al. Opt-impl: Scaling language model instruction meta learning through the lens of generalization. *arXiv preprint arXiv:2212.12017*, 2022.
- Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.
- Mario Michael Krell, Matej Kosec, Sergio P Perez, and Andrew Fitzgibbon. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance. *arXiv preprint arXiv:2107.02027*, 2021.
- Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, Daniel Haziza, Luca Wehrstedt, Jeremy Reizenstein, and Grigory Sizov. xformers: A modular and hackable transformer modelling library. <https://github.com/facebookresearch/xformers>, 2022.
- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. Dora: Weight-decomposed low-rank adaptation. *arXiv preprint arXiv:2402.09353*, 2024a.
- Tianqi Liu, Yao Zhao, Rishabh Joshi, Misha Khalman, Mohammad Saleh, Peter J Liu, and Jialu Liu. Statistical rejection sampling improves preference optimization. *arXiv preprint arXiv:2309.06657*, 2023.
- Tianqi Liu, Zhen Qin, Junru Wu, Jiaming Shen, Misha Khalman, Rishabh Joshi, Yao Zhao, Mohammad Saleh, Simon Baumgartner, Jialu Liu, et al. Lipo: Listwise preference optimization through learning-to-rank. *arXiv preprint arXiv:2402.01878*, 2024b.
- YanJun Ma, Dianhai Yu, Tian Wu, and Haifeng Wang. Paddlepaddle: An open-source deep learning platform from industrial practice. *Frontiers of Data and Computing*, 1(1):105–115, 2019.
- Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- M. Pagliardini, D. Paliotta, M. Jaggi, and F. Fleuret. Faster causal attention over large sequences through sparse flash attention. In *Proceedings of the international conference on Neural Information Processing Systems (NeurIPS)*, pp. 59808–59831, 2023. URL <https://fleuret.org/papers/pagliardini-et-al-neurips2023.pdf>.
- Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.
- Alec Radford. Improving language understanding by generative pre-training. 2018.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Y Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro von Werra, Cl  mentine Fourier, Nathan Habib, Nathan Sarrazin, Omar Sanseviero, Alexander M. Rush, and Thomas Wolf. Zephyr: Direct distillation of lm alignment, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005*, 2025.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297, 2020.

A APPENDIX

A.1 BACKWARD PASS ALGORITHM DETAILS

The detailed implementation of the FLASHMASK backward pass is presented in Algorithm 2. Similar to the forward pass, we precompute the maximum and minimum values of **LTS**, **LTE**, **UTS**, and **UTE**. These precomputed values $LTStart_j^{min}$, $LTStart_j^{max}$, $LTEnd_j^{min}$, $LTEnd_j^{max}$, $UTStart_j^{min}$, $UTStart_j^{max}$, $UTEnd_j^{min}$, $UTEnd_j^{max}$ can be directly loaded into registers and kept resident because the backward computation operates in a column-parallel mode. Additionally, **LTS**_{*j*}, **LTE**_{*j*}, **UTS**_{*j*}, and **UTE**_{*j*} can be loaded into SRAM outside of the inner loop (lines 10–11), thereby reducing the number of accesses to HBM to $4 \times N$. Within the inner loop, the computation logic of FLASHMASK remains identical to that of the forward pass.

A.2 END-TO-END TRAINING THROUGHPUT

Recent models such as Llama 3.1 and GPT-4, the Claude series, and Google’s Gemini support sequence modeling beyond 128K tokens. FLASHMASK, with its reduced memory overhead, facilitates training with even longer contexts. However, existing public DPO and RM datasets lack training data for scenarios exceeding 128K tokens. To comprehensively evaluate FLASHMASK, we constructed synthetic data to simulate long-sequence training and verify end-to-end throughput improvements. We validated our method across four downstream tasks involving the fine-tuning and alignment training of large language models: SFT, LoRA, DPO, and RM.

A.2.1 DATA CONSTRUCTION METHOD

For end-to-end training, to realistically simulate real dataset distributions, we needed to distinguish between source tokens and target tokens within a document’s sequence length. Additionally, the data construction method differed from that used in the kernel experiments. Given a maximum training sequence length N and a document count range $n \in [1, 10]$, we first randomly sampled the number of documents, then sampled each document’s sequence length such that the total sequence length equaled N . The last document was considered as padding. For the RM training, special constraints

Algorithm 2 FlashAttention-2 Backward Pass Extended with FLASHMASK

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r , vectors $\mathbf{LTS}, \mathbf{LTE}, \mathbf{UTS}, \mathbf{UTE} \in \mathbb{R}^N$.

- 1: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 2: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
- 3: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ into T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
- 4: Compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
- 5: Divide $\mathbf{LTS}, \mathbf{LTE}, \mathbf{UTS}, \mathbf{UTE}$ into T_c blocks $\mathbf{LTS}_1, \dots, \mathbf{LTS}_{T_c}, \mathbf{LTE}_1, \dots, \mathbf{LTE}_{T_c}, \mathbf{UTS}_1, \dots, \mathbf{UTS}_{T_c}, \mathbf{UTE}_1, \dots, \mathbf{UTE}_{T_c}$, of size B_c each.
- 6: Precompute the min and max row index $LTStart_j^{min} = \min(\mathbf{LTS}_j)$, $LTStart_j^{max} = \max(\mathbf{LTS}_j)$, $LTEnd_j^{min} = \min(\mathbf{LTE}_j)$, $LTEnd_j^{max} = \max(\mathbf{LTE}_j)$, $UTStart_j^{min} = \min(\mathbf{UTS}_j)$, $UTStart_j^{max} = \max(\mathbf{UTS}_j)$, $UTEnd_j^{min} = \min(\mathbf{UTE}_j)$, $UTEnd_j^{max} = \max(\mathbf{UTE}_j)$, $\forall j \in \{1, \dots, T_c\}$, write to HBM.
- 7: **for** $1 \leq j \leq T_c$ **do**
- 8: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 9: Initialize $\mathbf{dK}_j = (0)_{B_c \times d}$, $\mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
- 10: Load $\mathbf{LTS}_j$ and $\mathbf{LTE}_j$ from HBM to on-chip SRAM.
- 11: Load $\mathbf{UTS}_j$ and $\mathbf{UTE}_j$ from HBM to on-chip SRAM.
- 12: **for** $1 \leq i \leq T_r$ **do**
- 13: **if** $(i-1) \times B_r \geq LTStart_j^{max}$ **and** $i \times B_r \leq LTEnd_j^{min}$ **then**
- 14: Continue // lower triangular skip calculation of masked block
- 15: **end if**
- 16: **if** $(i-1) \times B_r \geq UTStart_j^{max}$ **and** $i \times B_r \leq UTEnd_j^{min}$ **then**
- 17: Continue // upper triangular skip calculation of masked block
- 18: **end if**
- 19: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.
- 20: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 21: **if** $i \times B_r > LTStart_j^{min}$ **and** $(i-1) \times B_r < LTEnd_j^{max}$ **then**
- 22: On chip, apply mask: $\mathbf{S}_i^{(j)}[x][y] = -\infty, \forall x, y$, such that $\mathbf{LTS}_j[y] \leq (i-1) \times B_r + x < \mathbf{LTE}_j[y]$
- 23: **end if**
- 24: **if** $i \times B_r > UTStart_j^{min}$ **and** $(i-1) \times B_r < UTEnd_j^{max}$ **then**
- 25: On chip, apply mask: $\mathbf{S}_i^{(j)}[x][y] = -\infty, \forall x, y$, such that $\mathbf{UTS}_j[y] \leq (i-1) \times B_r + x < \mathbf{UTE}_j[y]$
- 26: **end if**
- 27: On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
- 28: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
- 29: On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
- 30: On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
- 31: Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.
- 32: On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
- 33: **end for**
- 34: Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
- 35: **end for**
- 36: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

were applied: $n \in [1, 3]$ for $N \in (0, 4096]$ and $n \in [1, 4]$ for $N \in (4096, 8192]$. During sampling, we set the minimum document length to 128 for SFT, LoRA, and DPO, and 512 for RM. Padding lengths did not exceed 128 for SFT, LoRA, DPO, and 512 for RM.

Assuming a document had a sequence length L , it was further divided into source tokens and target tokens. SFT and LoRA were represented as $(Question, Answer)$ pairs, DPO as $(Question, Answer_1, Answer_2)$ with two answers, and RM, having 2 to 6 answers, was standard-

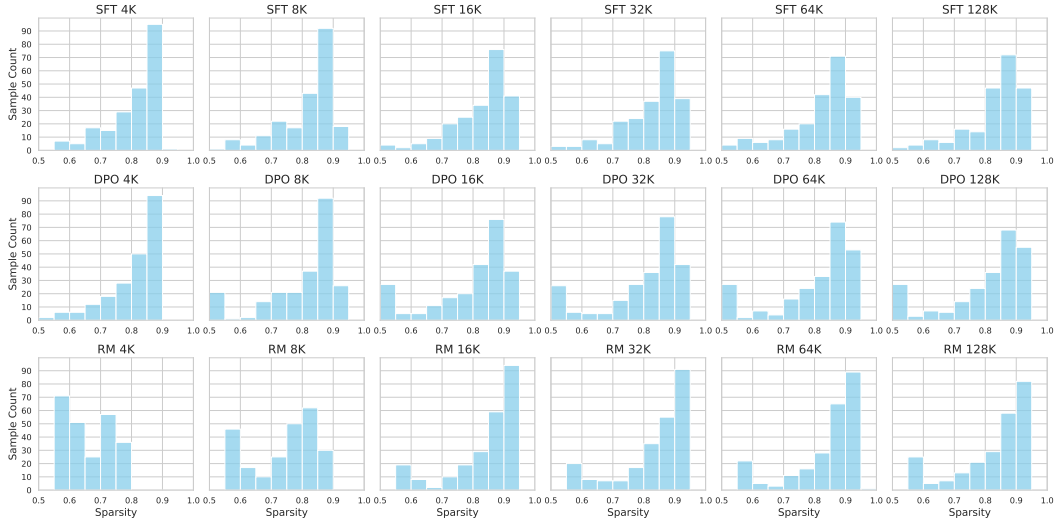


Figure 6: Sparsity distribution of synthetic dataset for end-to-end training throughput testing.

ized to have 6 answers: $(Question, Answer_1, \dots, Answer_6)$. Thus, L was partitioned into a query and k answers based on the training task, with k being 1 for SFT and LoRA, 2 for DPO, and 6 for RM. The length of each answer was randomly determined from the range $[\frac{0.1L}{1+0.1k}, \frac{0.2L}{1+0.2k}]$, making each answer approximately 10% to 20% of the query length. Consequently, the query length was calculated as L minus the total answer lengths. For each sequence length N , we collected 240 valid samples and categorized them into 10 bins by sparsity ρ , as illustrated in Figure 6.

Table 1: Hyperparameters and distributed configurations for various scales of Llama2 models.

Model	LLama2-7B	LLama2-13B	LLama2-70B
Batch Size	16	16	16
AccSteps	2	4	16
Sharding Stage1 Degree	8	4	1
Tensor Parallel Degree	4	4	8
PipeLine Parallel Degree	1	2	4
Sequence Parallel	✓	✓	✓

A.2.2 EXPERIMENTAL CONFIGURATION AND DISTRIBUTED STRATEGY

We evaluated different model scales of Llama2 (7B, 13B, 70B) across sequence lengths ranging from 4K to 544K, comparing against two dense methods: Vanilla Attention and FlashAttention DenseMask. All end-to-end throughput experiments were conducted on four servers, each equipped with eight NVIDIA A800-SXM 80G GPUs, totaling 32 GPUs. The objective was not to optimize peak performance for each configuration but to assess scalability with longer sequences; thus, we uniformly enabled full recomputation. Model parameters and computations utilized the BFloat16 data type, while gradient accumulation and communication employed Float32. The hyperparameters and distributed strategies for different scales are detailed in Table 1.

A.2.3 END-TO-END TRAINING MEMORY CONSUMPTION

Figure 2 in the main paper reports the end-to-end training throughput. We also recorded the peak memory consumption, presented in Figure 7. Notably, the memory usage of FLASHMASK increases significantly slower than that of dense methods. However, the figure also indicates that FLASHMASK’s memory consumption still escalates rapidly with longer sequence lengths, primarily due to increased activation memory from longer sequences, as shown in Table 2. The *Param & Opt State* column indicates the memory consumption for parameters, gradients, and optimizer states, with sharding stage 1 applied. *Activations* refers to the memory consumed by the inputs of the 32 decoder layers. *Peak Mem One Layer* represents the peak memory usage when full recompute is employed. *Total* denotes the overall memory consumption for FlashAttention without the attention mask.

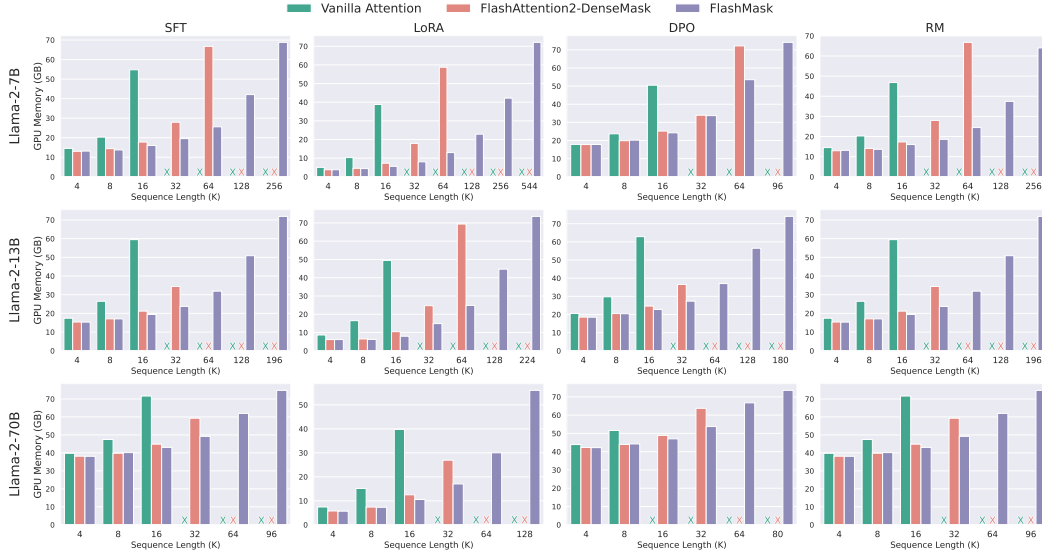


Figure 7: End-to-end training peak memory consumption across varying sequence lengths for different Llama2 model scales in four downstream training tasks: SFT, LoRA, DPO, and RM.

Table 2: Memory consumption comparison between FlashAttention without attention mask and FLASHMASK on the Llama-2 7B model. The observed differences in total memory footprint are attributed to memory fragmentation effects.

Sequence Length (K)	Param & Opt State	Activations	Peak Mem One Layer	Total	FLASHMASK
4	13.12	0.00	0.73	13.86	13.14
8	13.12	0.00	1.29	14.41	13.73
16	13.12	1.00	2.50	16.63	16.01
32	13.12	2.00	4.95	20.07	19.52
64	13.12	4.00	9.89	27.02	25.57
128	13.12	8.00	19.78	40.91	42.08
256	13.12	16.00	39.56	68.69	68.81

A.3 END-TO-END TRAINING CONVERGENCE VERIFICATION

We selected the Llama 3.1 8B model to verify convergence across four downstream tasks involving the fine-tuning and alignment training of large language models: SFT, LoRA, DPO, and RM. SFT and LoRA utilized the same dataset, validated using `allenai/tulu-v2-sft-mixture` Ivison et al. (2023). For DPO and RM, which both employ (*Question, Answer*) data formats, we used the `HuggingFaceH4/ultrafeedback_binarized` Tunstall et al. (2023) dataset for validation. We consistently applied a linear learning rate decay strategy, with warm-up steps set to 3% of the total training steps. The AdamW optimizer was used with $\beta_1 = 0.9$ and $\beta_2 = 0.999$. Model parameters and computations utilized the BFloat16 data type, while gradient accumulation and communication employed Float32. The maximum training sequence length was set to 8K. Distributed parallelism combined sharding and tensor parallelism. Additional hyperparameters are listed in Table 3.

A.4 SPARSITY-RELATED EXPERIMENTS

Prior to each run, we perform 10 warm-up iterations, followed by 100 runs of the kernel computation, recording the average execution time (in milliseconds) using CUDA Events. We aimed to verify that the computational complexity of FLASHMASK scales linearly with block sparsity in the attention mask. We report the total latency for the kernel’s forward and backward passes. This validation was performed on sequences of length 32K for three common mask types: *Causal Document Mask*, *Share Question Mask*, and *Document Mask*, corresponding to downstream training tasks in large language models such as SFT, DPO/RM, and pre-training of vision models like NaViT, respectively.

Table 3: The configuration of end-to-end training loss convergence verification.

Task	Dataset	Learning Rate	Training Step	Epochs	Batch Size	AccSteps	GPUs	Sharding Degree	TP Degree
SFT	tulu-v2-sft-mixture	2e-05	12000	3	16	1	32	16	2
LoRA	tulu-v2-sft-mixture	0.0002	12000	3	16	1	32	16	2
DPO	HuggingFaceH4/ultrafeedback_binarized	5e-07	4000	2	4	2	8	2	4
RM	HuggingFaceH4/ultrafeedback_binarized	1e-05	4000	2	4	1	8	4	2

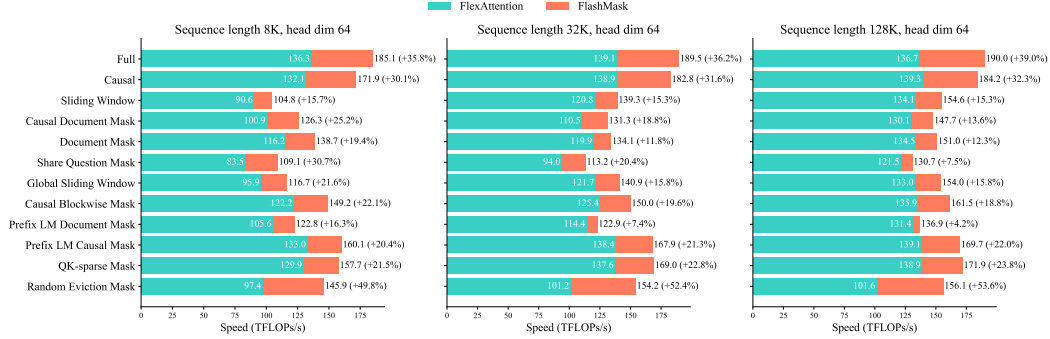


Figure 8: Kernel forward and backward speed (head dim 64, BF16) on A100-SXM 80G GPU. FlexAttention using PyTorch 2.6.0.dev20240920+cu124.

A.4.1 DATA CONSTRUCTION METHOD

Causal Document Mask and *Share Question Mask* are causal attention types with block sparsity values in the range [0.5, 1.0], while the *Document Mask* is a bidirectional attention type with block sparsity values in [0.0, 1.0]. We partitioned the sparsity values into buckets: 10 buckets for causal types and 20 buckets for bidirectional types, each with intervals of 0.05, ensuring the number of samples per bucket ranged between 10 and 20.

For the *Causal Document Mask*, given a maximum sequence length, we limited the number of documents to [2, 20]. We randomly sampled the number of documents and then sampled the length of each document such that the total sequence length equaled the maximum sequence length. Following each sample, we calculated the block sparsity and assigned it to the corresponding bucket until each bucket met the required number of samples. The data sampling process for the *Document Mask* was similar; to ensure coverage of all sparsity levels, the number of documents was limited to [2, 10].

The data sampling for the *Share Question Mask* differed slightly. The number of documents was limited to [1, 5]. We first sampled the length of each document to sum up to the given maximum sequence length. Each document was then partitioned into a *Question* and *Answers*, ensuring there was one *Question* and 2 to 6 *Answers*. As before, after each sample, we calculated the block sparsity and allocated it to the appropriate bucket until all buckets were adequately populated.

In total, we sampled 182 samples for the *Causal Document Mask*, 175 for the *Share Question Mask*, and 374 for the *Document Mask*.

A.5 KERNEL PERFORMANCE COMPARISON

A.5.1 TESTING METHOD

Both FLASHMASK and FlexAttention exploit sparsity in the attention mask to skip fully masked blocks, thereby reducing redundant computations. To provide an intuitive comparison, we employ the TFLOPs/s metric for evaluation. For each test case, we assess both the forward and backward computations. Prior to each run, we perform 10 warm-up iterations, followed by 100 runs of the kernel computation, recording the average execution time (in milliseconds) using CUDA Events. Based on the block sparsity in the attention mask, we calculate the FLOPs for a single run and subsequently compute the TFLOPs/s.

A.5.2 DATA CONSTRUCTION METHOD

We conducted detailed comparisons across varying batch sizes and sequence lengths (8K, 32K, 128K), different head dimensions (64, 128), and numbers of heads. We fixed the total number of tokens at

Table 4: Kernel speed details (8K, head dim 128, BF16) on A100-SXM 80G GPU.

Method	Operation	FW Time (ms)	BW Time (ms)	TOTAL Time (ms)	FW TFLOPs	BW TFLOPs	TOTAL TFLOPs	FW TFLOPs/s	BW TFLOPs/s	TOTAL TFLOPs/s	Sparsity
FlexAttention	Full	109.77	331.80	441.57	17.59	43.98	61.57	160.27	132.55	139.44	0.00
	Causal	55.61	179.82	235.43	8.93	22.33	31.27	160.65	124.20	132.81	0.49
	Sliding Window	10.20	41.85	52.05	1.33	3.33	4.66	130.50	79.54	89.53	0.92
	Causal Document Mask	17.57	62.99	80.56	2.42	6.05	8.47	136.79	95.10	104.16	0.86
	Document Mask	31.01	103.90	134.90	4.54	11.34	15.87	143.88	106.67	115.17	0.74
	Share Question Mask	13.17	50.07	63.24	1.62	4.05	5.68	122.39	80.38	89.11	0.91
	Global Sliding Window	21.65	79.82	101.47	2.89	7.24	10.13	133.74	90.67	99.85	0.84
	Causal Blockwise Mask	38.44	125.16	163.60	5.75	14.37	20.12	149.19	114.42	122.58	0.67
	Prefix LM Document Mask	18.92	66.00	84.92	2.53	6.32	8.85	132.59	94.89	103.27	0.86
	Prefix LM Causal Mask	69.06	218.47	287.53	11.06	27.66	38.72	160.20	126.61	134.68	0.37
	QK-sparse Mask	53.17	170.11	223.28	8.36	20.91	29.28	157.30	122.92	131.10	0.52
	Random Eviction Mask	67.53	215.56	283.09	8.93	22.33	31.27	132.30	103.61	110.45	0.49
FLASHMASK	Full	76.30	224.34	300.64	17.59	43.98	61.57	230.56	196.05	204.81	0.00
	Causal	39.01	118.60	157.61	8.93	22.33	31.27	229.02	188.31	198.39	0.49
	Sliding Window	8.83	30.59	39.41	1.33	3.33	4.66	150.84	108.83	118.24	0.92
	Causal Document Mask	13.78	44.24	58.03	2.42	6.05	8.47	174.26	135.48	144.67	0.86
	Document Mask	25.12	73.76	98.88	4.56	11.40	15.96	177.95	151.74	158.40	0.74
	Share Question Mask	9.83	33.87	43.70	1.62	4.05	5.68	163.93	118.89	129.01	0.91
	Global Sliding Window	20.10	53.07	73.17	2.89	7.24	10.13	144.05	136.36	138.47	0.84
	Causal Blockwise Mask	30.76	86.04	116.80	5.75	14.37	20.12	186.37	166.59	171.79	0.67
	Prefix LM Document Mask	16.04	46.82	62.86	2.53	6.32	8.85	156.17	133.89	139.58	0.86
	Prefix LM Causal Mask	55.20	162.31	217.51	11.06	27.66	38.72	200.42	170.42	178.03	0.37
	QK-sparse Mask	43.57	120.84	164.41	8.44	21.11	29.55	193.80	174.67	179.74	0.52
	Random Eviction Mask	49.04	135.06	184.10	8.93	22.33	31.27	182.17	165.36	169.84	0.49

Table 5: Kernel speed details (32K, head dim 128, BF16) on A100-SXM 80G GPU.

Method	Operation	FW Time (ms)	BW Time (ms)	TOTAL Time (ms)	FW TFLOPs	BW TFLOPs	TOTAL TFLOPs	FW TFLOPs/s	BW TFLOPs/s	TOTAL TFLOPs/s	Sparsity
FlexAttention	Full	434.91	1296.22	1731.12	70.37	175.92	246.29	161.80	135.72	142.27	0.00
	Causal	214.09	667.11	881.20	35.32	88.30	123.63	164.98	132.37	140.29	0.50
	Sliding Window	29.40	101.77	131.18	4.53	11.32	15.84	153.95	111.20	120.79	0.94
	Causal Document Mask	25.40	87.30	112.70	3.68	9.19	12.86	144.54	105.08	113.97	0.95
	Document Mask	39.29	126.59	165.88	5.72	14.29	20.01	143.78	111.30	118.98	0.92
	Share Question Mask	63.59	81.98	145.57	2.30	5.75	8.04	131.21	90.15	99.00	0.97
	Global Sliding Window	61.53	198.82	260.36	9.29	23.23	32.52	151.01	116.84	124.92	0.87
	Causal Blockwise Mask	59.26	187.03	246.29	8.94	22.35	31.29	150.30	118.95	126.48	0.87
	Prefix LM Document Mask	27.78	91.34	119.11	3.83	9.58	13.41	137.70	104.65	112.36	0.95
	Prefix LM Causal Mask	269.34	828.90	1098.24	44.05	110.12	154.17	163.55	132.85	140.38	0.37
	QK-sparse Mask	209.04	644.08	853.12	33.99	84.97	118.95	162.58	131.92	139.43	0.52
	Random Eviction Mask	261.30	810.44	1071.74	35.32	88.30	123.63	135.18	108.96	115.35	0.50
FLASHMASK	Full	304.25	860.73	1164.98	70.37	175.92	246.29	231.28	204.39	211.41	0.00
	Causal	153.25	430.64	583.89	35.32	88.30	123.63	230.49	205.05	211.73	0.50
	Sliding Window	28.50	72.25	100.76	4.53	11.32	15.84	158.83	156.63	157.25	0.94
	Causal Document Mask	24.76	60.51	85.27	3.68	9.19	12.86	148.12	151.60	150.59	0.95
	Document Mask	41.76	89.84	131.60	5.77	14.42	20.19	134.92	158.68	150.84	0.92
	Share Question Mask	18.28	42.77	61.05	2.30	5.75	8.04	125.41	134.07	131.47	0.97
	Global Sliding Window	65.36	140.86	206.21	9.29	23.23	32.52	142.17	164.92	157.71	0.87
	Causal Blockwise Mask	54.29	127.06	181.34	8.94	22.35	31.29	163.41	175.15	171.61	0.87
	Prefix LM Document Mask	33.37	64.23	97.59	3.83	9.58	13.41	114.50	148.85	137.07	0.95
	Prefix LM Causal Mask	217.99	606.89	824.88	44.05	110.12	154.17	202.07	181.45	186.90	0.37
	QK-sparse Mask	173.04	446.80	619.84	34.09	85.23	119.33	197.02	190.51	192.51	0.52
	Random Eviction Mask	192.18	494.40	686.59	35.32	88.30	123.63	183.79	178.61	180.06	0.50

128K; by varying the sequence length, we computed the corresponding batch size. With the hidden size fixed at 4096, varying the head dimension allowed us to determine the number of heads.

To encompass a broader range of block sparsity cases in the attention mask for a given sequence length, we utilized constructed data for testing. Given a test sequence length, we defined the document count range as $n \in [\text{Doc}_{\min}, \text{Doc}_{\max}]$. We first sampled the number of documents and then sampled the length of each document such that the total length equaled the test sequence length. For the *Share Question Mask* type, we further partitioned each document into one *Question* and 2 to 6 *Answers*. The document count ranges were [3, 7] for 8K, [10, 14] for 32K, and [11, 15] for 128K. For each sequence length, we generated five test data samples.

Table 6: Kernel speed details (128K, head dim 128, BF16) on A100-SXM 80G GPU.

Method	Operation	FW Time (ms)	BW Time (ms)	TOTAL Time (ms)	FW TFLOPs	BW TFLOPs	TOTAL TFLOPs	FW TFLOPs/s	BW TFLOPs/s	TOTAL TFLOPs/s	Sparsity
FlexAttention	Full	1726.14	5241.59	6967.73	281.48	703.69	985.16	163.07	134.25	141.39	0.00
	Causal	853.26	2647.03	3500.29	140.88	352.19	493.06	165.10	133.05	140.86	0.50
	Sliding Window	106.38	340.59	446.97	17.31	43.27	60.58	162.71	127.05	135.54	0.94
	Causal Document Mask	80.71	258.22	338.93	12.82	32.05	44.87	158.78	124.03	132.30	0.95
	Document Mask	159.54	494.81	654.35	25.68	64.21	89.89	160.44	129.20	136.81	0.91
	Share Question Mask	48.41	159.07	207.49	7.41	18.52	25.92	152.91	116.31	124.85	0.97
	Global Sliding Window	216.10	664.58	880.69	34.86	87.14	122.00	161.30	131.13	138.53	0.88
	Causal Blockwise Mask	221.27	671.55	892.82	35.32	88.29	123.60	159.60	131.46	138.44	0.87
	Prefix LM Document Mask	85.82	267.09	352.91	13.39	33.48	46.87	155.98	125.26	132.73	0.95
	Prefix LM Causal Mask	1073.03	3309.05	4382.08	175.99	439.98	615.97	164.01	132.96	140.56	0.37
	QK-sparse Mask	831.00	2556.67	3387.67	136.06	340.15	476.22	163.73	133.05	140.57	0.52
	Random Eviction Mask	1037.20	3318.50	4355.70	140.88	352.19	493.06	135.82	106.13	113.20	0.50
FLASHMASK	Full	1216.27	3403.06	4619.33	281.48	703.69	985.16	231.42	206.78	213.27	0.00
	Causal	631.12	1679.32	2310.44	140.88	352.19	493.06	223.21	209.72	213.41	0.50
	Sliding Window	105.78	238.96	344.74	17.31	43.27	60.58	163.63	181.09	175.73	0.94
	Causal Document Mask	87.46	179.89	267.35	12.82	32.05	44.87	146.27	178.03	167.61	0.95
	Document Mask	177.02	538.45	715.47	25.72	64.29	90.01	142.00	178.59	165.71	0.91
	Share Question Mask	62.65	109.80	172.44	7.41	18.52	25.92	118.01	168.51	150.12	0.97
	Global Sliding Window	248.39	482.82	731.21	34.86	87.14	122.00	140.33	180.49	166.85	0.88
	Causal Blockwise Mask	210.42	464.94	675.36	35.32	88.29	123.60	167.81	189.88	183.00	0.87
	Prefix LM Document Mask	121.43	193.09	314.52	13.39	33.48	46.87	110.01	173.27	148.75	0.95
	Prefix LM Causal Mask	891.90	2381.23	3273.13	175.99	439.98	615.97	197.32	184.77	188.19	0.37
	QK-sparse Mask	702.86	1748.06	2450.92	136.16	340.40	476.56	193.73	194.73	194.44	0.52
	Random Eviction Mask	776.92	1933.23	2710.16	140.88	352.19	493.06	181.32	182.18	181.93	0.50

A.5.3 EXPERIMENTAL RESULTS

Figure 8 presents the comparison of total TFLOPs/s for forward and backward passes between FLASHMASK and FlexAttention when the head dimension is 64. Similar results are illustrated in Figure 5 in the main paper for a head dimension of 128. Across all cases, FLASHMASK outperforms FlexAttention in terms of total TFLOPs/s for both forward and backward passes, with improvements

Table 7: Kernel speed details (8K, head dim 64, BF16) on A100-SXM 80G GPU.

Method	Operation	FW Time (ms)	BW Time (ms)	TOTAL Time (ms)	FW TFLOPs	BW TFLOPs	TOTAL TFLOPs	FW TFLOPs/s	BW TFLOPs/s	TOTAL TFLOPs/s	Sparsity
FlexAttention	Full	111.99	339.81	451.80	17.59	43.98	61.57	157.09	129.43	136.28	0.00
	Causal	56.41	180.30	236.71	8.93	22.33	31.27	158.37	123.87	132.09	0.49
	Sliding Window	10.69	40.74	51.43	1.33	3.33	4.66	124.51	81.71	90.61	0.92
	Causal Document Mask	18.03	65.14	83.17	2.42	6.05	8.47	133.31	91.92	100.85	0.86
	Document Mask	31.17	102.82	133.98	4.54	11.34	15.87	143.68	107.97	116.20	0.74
	Share Question Mask	13.79	53.84	67.63	1.62	4.06	5.68	117.11	74.87	83.46	0.91
	Global Sliding Window	23.83	81.80	105.63	2.89	7.24	10.13	121.49	88.47	95.92	0.84
	Causal Blockwise Mask	39.45	124.74	164.18	5.75	14.37	20.12	145.32	114.83	122.15	0.67
	Prefix LM Document Mask	18.49	64.64	83.13	2.53	6.32	8.85	135.95	96.92	105.57	0.86
	Prefix LM Causal Mask	70.71	220.50	291.21	11.06	27.66	38.72	156.47	125.44	132.98	0.37
	QK-sparse Mask	54.11	171.32	225.43	8.36	20.91	29.28	154.57	122.05	129.86	0.52
	Random Eviction Mask	75.03	246.11	321.14	8.93	22.33	31.27	119.07	90.75	97.36	0.49
FLASHMASK	Full	87.04	245.56	332.59	17.59	43.98	61.57	202.13	179.10	185.13	0.00
	Causal	45.31	136.58	181.89	8.93	22.33	31.27	197.18	163.52	171.90	0.49
	Sliding Window	10.39	34.06	44.45	1.33	3.33	4.66	128.14	97.73	104.84	0.92
	Causal Document Mask	16.08	50.34	66.42	2.42	6.05	8.47	149.08	119.03	126.31	0.86
	Document Mask	28.11	84.48	112.59	4.56	11.40	15.96	158.54	132.11	138.71	0.74
	Share Question Mask	12.69	39.06	51.75	1.62	4.06	5.68	127.11	103.22	109.07	0.91
	Global Sliding Window	23.68	63.17	86.85	2.89	7.24	10.13	122.25	114.57	116.66	0.84
	Causal Blockwise Mask	34.22	100.23	134.44	5.75	14.37	20.12	167.38	142.95	149.17	0.67
	Prefix LM Document Mask	17.99	53.44	71.42	2.53	6.32	8.85	139.18	117.24	122.77	0.86
	Prefix LM Causal Mask	59.65	182.22	241.87	11.06	27.66	38.72	156.47	131.79	160.10	0.37
	QK-sparse Mask	47.46	139.89	187.35	8.44	21.11	29.55	177.89	150.89	157.73	0.52
	Random Eviction Mask	57.58	156.77	214.35	8.93	22.33	31.27	155.15	142.47	145.87	0.49

ranging from 4.2% to 53.6%. FLASHMASK achieves 33.6% to 55.1% of the theoretical maximum FLOPs/s on the A100 GPU. Tables 4 – 9 detail, for each test mask case, the sparsity, and the forward and backward computation latency, TFLOPs, and TFLOPs/s.

Table 8: Kernel speed details (32K, head dim 64, BF16) on A100-SXM 80G GPU.

Method	Operation	FW Time (ms)	BW Time (ms)	TOTAL Time (ms)	FW TFLOPs	BW TFLOPs	TOTAL TFLOPs	FW TFLOPs/s	BW TFLOPs/s	TOTAL TFLOPs/s	Sparsity
FlexAttention	Full	445.55	1325.12	1770.67	70.37	175.92	246.29	157.94	132.76	139.09	0.00
	Causal	218.23	671.98	890.21	35.32	88.30	123.63	161.85	131.41	138.87	0.50
	Sliding Window	30.15	100.98	131.13	4.53	11.32	15.84	150.13	112.08	120.83	0.94
	Causal Document Mask	26.17	90.05	116.22	3.68	9.19	12.86	140.30	101.85	110.50	0.95
	Document Mask	38.89	125.98	164.86	5.72	14.29	20.01	145.94	111.98	119.94	0.92
	Share Question Mask	18.75	69.39	88.14	2.37	5.93	8.31	126.37	85.32	94.04	0.97
	Global Sliding Window	64.43	202.92	267.34	9.29	23.23	32.52	144.23	114.48	121.65	0.87
	Causal Blockwise Mask	61.11	187.21	248.32	8.94	22.35	31.29	145.71	118.84	125.45	0.87
	Prefix LM Document Mask	26.91	90.11	117.02	3.83	9.58	13.41	142.18	106.09	114.39	0.95
	Prefix LM Causal Mask	278.20	836.05	1114.25	44.05	110.12	154.17	158.33	131.72	138.36	0.37
	QK-sparse Mask	215.41	648.92	864.33	33.99	84.97	118.95	157.77	130.93	137.62	0.52
	Random Eviction Mask	292.99	928.88	1221.87	35.32	88.30	123.63	120.56	95.07	101.18	0.50
FLASHMASK	Full	346.27	953.51	1299.78	70.37	175.92	246.29	203.22	184.50	189.49	0.00
	Causal	175.61	500.70	676.31	35.32	88.30	123.63	201.13	176.36	182.79	0.50
	Sliding Window	32.34	81.43	113.77	4.53	11.32	15.84	139.97	138.98	139.27	0.94
	Causal Document Mask	28.69	69.11	97.80	3.68	9.19	12.86	127.83	132.73	131.29	0.95
	Document Mask	44.79	103.14	147.93	5.77	14.42	20.19	125.80	138.00	134.12	0.92
	Share Question Mask	22.92	50.30	73.22	2.37	5.93	8.31	103.30	117.71	113.19	0.97
	Global Sliding Window	69.46	161.42	230.88	9.29	23.23	32.52	133.78	143.91	140.86	0.87
	Causal Blockwise Mask	59.67	147.83	207.50	8.94	22.35	31.29	148.59	150.60	149.98	0.87
	Prefix LM Document Mask	35.43	73.44	108.87	3.83	9.58	13.41	107.83	130.17	122.87	0.95
	Prefix LM Causal Mask	234.73	683.57	918.30	44.05	110.12	154.17	187.66	161.10	167.89	0.37
	QK-sparse Mask	185.20	520.75	705.95	34.09	85.23	119.33	184.09	163.67	169.03	0.52
	Random Eviction Mask	223.56	578.42	801.98	35.32	88.30	123.63	158.00	152.66	154.15	0.50

Table 9: Kernel speed details (128K, head dim 64, BF16) on A100-SXM 80G GPU.

Method	Operation	FW Time (ms)	BW Time (ms)	TOTAL Time (ms)	FW TFLOPs	BW TFLOPs	TOTAL TFLOPs	FW TFLOPs/s	BW TFLOPs/s	TOTAL TFLOPs/s	Sparsity
FlexAttention	Full	1779.78	5424.93	7204.72	281.48	703.69	985.16	158.15	129.71	136.74	0.00
	Causal	873.21	2666.96	3540.16	140.88	352.19	493.06	165.33	132.06	139.28	0.50
	Sliding Window	108.08	343.79	451.88	17.31	43.27	60.58	160.14	125.87	134.06	0.94
	Causal Document Mask	83.21	261.48	344.69	12.82	32.05	44.87	154.02	122.48	130.09	0.95
	Document Mask	164.96	501.19	666.15	25.68	64.21	89.89	155.36	127.63	134.49	0.91
	Share Question Mask	50.28	162.70	212.97	7.40	18.51	25.91	147.12	113.60	121.51	0.97
	Global Sliding Window	228.79	688.21	917.00	34.86	87.14	122.00	152.35	126.62	133.04	0.88
	Causal Blockwise Mask	226.86	682.44	909.31	35.32	88.29	123.60	155.66	129.37	135.93	0.87
	Prefix LM Document Mask	87.26	269.21	356.47	13.39	33.48	46.87	153.43	124.29	131.42	0.95
	Prefix LM Causal Mask	1093.12	3335.61	4428.74	175.99	439.98	615.97	161.00	131.90	139.08	0.37
	QK-sparse Mask	846.40	2583.14	3429.54	136.06	340.15	476.22	160.75	131.68	138.86	0.52
	Random Eviction Mask	1167.58	3683.61	4851.20	140.88	352.19	493.06	120.66	95.61	101.64	0.50
FLASHMASK	Full	1383.21	3800.81	5184.02	281.48	703.69	985.16	203.49	185.14	190.04	0.00
	Causal	706.68	1970.00	2676.68	140.88	352.19	493.06	199.35	178.78	184.21	0.50
	Sliding Window	120.53	271.26	391.79	17.31	43.27	60.58	143.60	159.52	154.63	0.94
	Causal Document Mask	98.31	204.97	303.28	12.82	32.05	44.87	130.11	156.25	147.75	0.95
	Document Mask	182.42	406.31	588.74	25.72	64.29	90.01	137.92	157.52	150.97	0.91
	Share Question Mask	72.72	125.14	197.87	7.40	18.51	25.91	101.51	147.72	130.66	0.97
	Global Sliding Window	243.70	548.38	792.08	34.86	87.14	122.00	143.03	158.91	154.03	0.88
	Causal Blockwise Mask	226.04	539.11	765.14	35.32	88.29	123.60	156.21	163.76	161.53	0.87
	Prefix LM Document Mask	123.12	218.68	341.80	13.39	33.48	46.87	108.50	153.00	136.90	0.95
	Prefix LM Causal Mask	943.12	2686.99	3630.11	175.99	439.98	615.97	186.60	163.74	169.68	0.37
	QK-sparse Mask	736.38	2036.58	2772.96	136.16	340.40	476.56	184.91	167.14	171.86	0.52
	Random Eviction Mask	893.22	2265.78	3159.00	140.88	352.19	493.06	157.72	155.44	156.08	0.50

B FLASHMASK APPLICATION IN INFERENCE

In the main body of our paper, we focused on the application of FLASHMASK during the training phase of large-scale models. However, it is important to highlight that FLASHMASK is equally effective during the inference stage. In this appendix, we provide detailed experimental results demonstrating the efficacy of FLASHMASK in inference, comparing it with state-of-the-art attention implementations, including FlashInfer Ye et al. (2025).

B.1 EXPERIMENTAL SETUP

Our experiments were conducted on an NVIDIA A100-SXM 80G GPU using FlashInfer version 0.1.6, CUDA 12.1, PyTorch 2.4, and BF16 data type. We set the batch size to 1, with 32 query/output

Table 10: Performance Comparison on **Causal Document Mask** at 8K, 32K, and 128K Sequence Lengths

Method	Seq Length	Sparsity	FW Time (ms)	FW TFLOPs	FW TFLOPs/s
FlashInfer SparseMask	8,192	0.8806	9.33	0.1313	13.95
FlashInfer DenseMask	8,192	0.8806	11.93	0.1313	11.01
FLASHMASK	8,192	0.8806	0.96	0.1313	135.07
FlashInfer SparseMask	32,768	0.9532	54.77	0.8233	15.01
FlashInfer DenseMask	32,768	0.9532	184.20	0.8233	4.47
FLASHMASK	32,768	0.9532	5.99	0.8233	137.09
FlashInfer SparseMask	131,072	0.9558	788.94	12.4435	15.77
FlashInfer DenseMask	131,072	0.9558	2,948.23	12.4435	4.22
FLASHMASK	131,072	0.9558	84.13	12.4435	147.58

Table 11: Performance Comparison on **Shared Question Mask** at 8K, 32K, and 128K Sequence Lengths

Method	Seq Length	Sparsity	FW Time (ms)	FW TFLOPs	FW TFLOPs/s
FlashInfer SparseMask	8,192	0.9324	6.12	0.0743	11.76
FlashInfer DenseMask	8,192	0.9324	11.94	0.0743	6.23
FLASHMASK	8,192	0.9324	0.73	0.0743	98.49
FlashInfer SparseMask	32,768	0.9742	32.87	0.4537	13.74
FlashInfer DenseMask	32,768	0.9742	184.40	0.4537	2.46
FLASHMASK	32,768	0.9742	4.59	0.4537	98.26
FlashInfer SparseMask	131,072	0.9751	443.80	7.0146	15.80
FlashInfer DenseMask	131,072	0.9751	2,948.89	7.0146	2.38
FLASHMASK	131,072	0.9751	61.57	7.0146	113.21

heads and 8 key/value heads, each with a head dimension of 128. The evaluation included typical attention masks such as the *Causal Document Mask*, *Document Mask*, and *Shared Question Mask*.

To ensure compatibility with FlashInfer’s sparse mask representation (where the mask block size $C = 64$), we adapted the datasets from Section A.5.2 so that each sub-document sequence length is divisible by 64. We defined the *mask block size* based on FlashInfer’s Block Sparse Row (BSR) API parameters R and C , and matched the *tiling block size* to the kernel’s operational dimensions.

B.2 COMPARISON WITH FLASHINFER

We compared FLASHMASK with FlashInfer’s dense mask API (`single_prefill_with_kv_cache`) and sparse mask API (`BlockSparseAttentionWrapper`) across various sequence lengths (8K, 32K, and 128K tokens). The results are summarized in Tables 10 to 14.

Efficiency Analysis: FLASHMASK consistently outperformed both the dense and sparse implementations of FlashInfer in terms of TFLOPs/s, particularly addressing the inefficiencies observed with FlashInfer’s dense mask API. While FlashInfer with sparse masks showed performance gains with increasing mask block sizes ($R, C \geq 16$), such large block sizes are seldom practical due to the nature of attention patterns in real-world applications.

In the FlashInfer `single_prefill_with_kv_cache` implementation (see `prefill.cuh` lines 1234–1241³), token-by-token dense masks lead to significant inefficiencies by performing unnecessary computations on fully masked blocks. Additionally, in FlashInfer’s `BlockSparseAttentionWrapper`, smaller mask block sizes increase the padded batch size (`nblks(padded_batch_size, 1, num_kv_heads)`), negatively impacting performance due to suboptimal kernel hyper-parameter tuning. In contrast, FLASHMASK efficiently computes only the required tiling blocks, avoiding redundant calculations, and thus achieves superior TFLOPs/s.

³<https://github.com/flashinfer-ai/flashinfer/blob/v0.1.6/include/flashinfer/attention/prefill.cuh#L1234-L1241>

B.3 IMPACT OF MASK BLOCK SIZE

For the *Document Mask*, we investigated the effect of varying the mask block size (R/C) on performance. Although FlashInfer DenseMask and FLASHMASK do not utilize specific R/C values, we included them in the comparison for completeness. The results for sequence lengths of 8K, 32K, and 128K tokens are presented in Tables 12, 13, and 14, respectively.

Table 12: Performance on **Document Mask** at 8K Sequence Length with Varying Mask Block Sizes

Method	R/C	Sparsity	FW Time (ms)	FW TFLOPs	FW TFLOPs/s
FlashInfer SparseMask	1	0.7868	15.39	0.2344	15.19
FlashInfer SparseMask	2	0.7613	8.57	0.2624	30.48
FlashInfer SparseMask	4	0.7613	4.31	0.2624	60.57
FlashInfer SparseMask	8	0.7613	3.23	0.2624	80.97
FlashInfer SparseMask	16	0.7613	1.65	0.2624	158.55
FlashInfer SparseMask	32	0.7613	1.51	0.2624	172.61
FlashInfer SparseMask	64	0.7613	1.51	0.2624	172.82
FlashInfer DenseMask	–	0.7613	11.91	0.2624	22.03
FLASHMASK	–	0.7613	1.66	0.2624	156.82

Table 13: Performance on **Document Mask** at 32K Sequence Length with Varying Mask Block Sizes

Method	R/C	Sparsity	FW Time (ms)	FW TFLOPs	FW TFLOPs/s
FlashInfer SparseMask	1	0.9064	104.65	1.6460	15.73
FlashInfer SparseMask	2	0.9064	52.46	1.6460	31.36
FlashInfer SparseMask	4	0.9064	25.96	1.6460	63.47
FlashInfer SparseMask	8	0.9064	19.68	1.6460	83.59
FlashInfer SparseMask	16	0.9064	9.87	1.6460	166.58
FlashInfer SparseMask	32	0.9064	8.89	1.6460	185.13
FlashInfer SparseMask	64	0.9064	8.89	1.6460	185.16
FlashInfer DenseMask	–	0.9064	183.99	1.6460	8.95
FLASHMASK	–	0.9064	11.73	1.6460	139.84

Table 14: Performance on **Document Mask** at 128K Sequence Length with Varying Mask Block Sizes

Method	R/C	Sparsity	FW Time (ms)	FW TFLOPs	FW TFLOPs/s
FlashInfer SparseMask	1	0.9116	1,571.12	24.8848	15.84
FlashInfer SparseMask	2	0.9116	783.62	24.8848	31.75
FlashInfer SparseMask	4	0.9116	391.20	24.8848	63.61
FlashInfer SparseMask	8	0.9116	288.97	24.8848	86.11
FlashInfer SparseMask	16	0.9116	145.13	24.8848	171.45
FlashInfer SparseMask	32	0.9116	131.31	24.8848	189.50
FlashInfer SparseMask	64	0.9116	131.33	24.8848	189.48
FlashInfer DenseMask	–	0.9116	2,946.81	24.8848	8.44
FLASHMASK	–	0.9116	172.81	24.8848	143.68